Fine-Tune It

LoRA, QLoRA, SFT and Preference Tuning

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

November 11, 2026

Table of Contents

1. Motivation: where prompting stopped; the map of the lecture
2. Prompt, retrieve or fine-tune: what fine-tuning changes and what it does not
3. LoRA: a frozen model and a small trained update, counted by hand; rank and target layers, measured
4. QLoRA: a 4-bit base with LoRA on top; what trains on a T4, measured
5. Supervised fine-tuning with TRL: data, the loss on the answer, settings; before and after, measured
6. Preference tuning: RLHF in three steps; DPO in practice, measured
7. Rewards that can be checked: GRPO; reward hacking
8. Data from a larger model: labels, distillation, synthetic data
9. Shipping: which method for which goal; merge, export, serve; a checklist
10. Summary and references

Prompting Reached Its Limit; a Small Add-On Fits in Memory

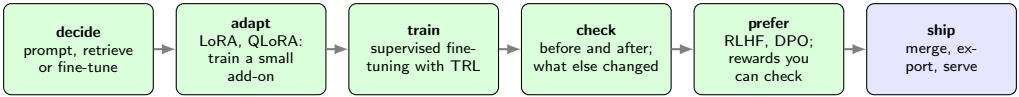
Qwen3-0.6B, prompt	English	Arabic	tokens
intent names only	42%	27%	79
names with definitions	58%	40%	176
10 random examples	77%	63%	291
10 most similar examples	93%	92%	270

100 test requests per language from MASSIVE (voice-assistant requests), the same requests in English and Arabic. Computed for this course.

Training memory, Qwen3-0.6B: full fine-tuning 9.5 GB; a LoRA add-on 2.5 GB.

- The best prompt needs a search for similar examples and 270 tokens instead of 79 on every request: a slower first token and a higher bill.
- Without the right examples, accuracy falls to 42% in English and 27% in Arabic.
- **The question of this lecture:** can we put the examples into the weights, so that the short prompt is enough?
- Training only a small add-on keeps memory low enough for one GPU.

The Map: Decide, Adapt, Train, Check, Prefer, Ship



- Each step fixes a problem the step before leaves. Red: the problem it fixes.
- Throughout: Qwen3-0.6B on the voice-assistant task in English and Arabic, and one T4 GPU with 16 GB of memory.

Learning Outcomes

By the end of this lecture you should be able to:

- **Decide** whether a problem needs a better prompt, retrieval or fine-tuning.
- **Count** the parameters LoRA trains, by hand, and **choose** its rank and target layers; **plan** a QLoRA run that fits a T4.
- **Fine-tune** a small model with TRL on your own examples, with the loss on the answers only.
- **Evaluate** a fine-tuned model before and after, on the task and on everything else.
- **Explain** RLHF in three steps, and **train** with DPO on preference pairs; **write** a reward function for GRPO.
- **Use** a larger model to label data, and **ship** the result: merge, export and serve.
- **Prerequisite knowledge:** decoder-only transformers; chat templates and prompting; training with AdamW in PyTorch; quantization and GPU memory.

Prompt, Retrieve or Fine-Tune?

what goes wrong	first fix	example
the model does not know your facts, or they change	retrieval: put the documents in the prompt	policies, prices, a product manual
the task is clear but the answers vary in format or style	a better prompt, with examples	JSON with fixed fields; a house style
a narrow task, repeated thousands of times, where prompts got long	supervised fine-tuning of a small model	labelling requests; extracting fields
a quality that is hard to write down but easy to judge	preference tuning (DPO)	tone, length, refusing politely
a task whose answer a program can check	reinforcement learning with rewards (GRPO)	arithmetic, code that passes tests, valid JSON

- **Always first:** a test set with known answers and a prompt baseline. Without them, nobody can tell whether training helped.
- The fixes combine: a fine-tuned model can still read retrieved documents.

Fine-Tuning Teaches Behaviour More Easily Than Facts

- **What fine-tuning does well:** a format, a style, a language, a narrow task, a fixed set of labels. The model already has the knowledge; training makes it use it the way you want, without long instructions.
- **What it does poorly:** new facts. Gekhman et al. found that examples with knowledge the model lacks are learned slowly and, once learned, make the model hallucinate more.
- Ovadia et al. compared both routes for adding knowledge: retrieval did better than fine-tuning.

Gekhman et al., EMNLP 2024, arXiv:2405.05904v3; Ovadia et al., 2023, arXiv:2312.05934v3.

Rule of thumb

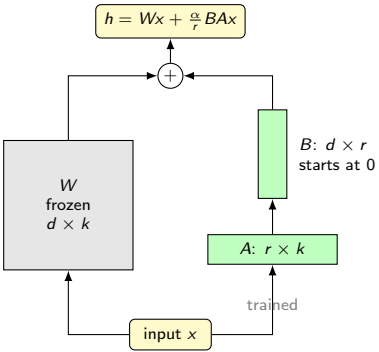
Facts that change, or that you must cite: **retrieve**.

How to answer: **fine-tune**.

Both: fine-tune the behaviour, retrieve the facts.

- Our running task is behaviour: the model knows what "wake me up at nine" means; it must learn to answer with the label `alarm_set` and nothing else.

LoRA: Freeze the Weights, Train a Thin Update



- **The problem:** full fine-tuning needs a gradient and two AdamW states for every weight: 16 bytes per parameter.
- **The fix:** keep W frozen and learn a change BA of low rank r. A and B are thin: r is 8 to 64, while d and k are in the thousands.
- B starts at zero, so training starts from exactly the original model.
- α/r scales the update. After training, BA can be added into W: the model is as fast as before.

Hu et al., ICLR 2022, arXiv:2106.09685v2.

LoRA by Hand: One Layer of Qwen3-0.6B

matrix	$k \to d$	**full** $d \times k$	**LoRA** $r(k+d)$
`q_proj`	$1024 \to 2048$	2,097,152	49,152
`k`, `v`	$1024 \to 1024$	$2\times$ 1,048,576	$2\times$ 32,768
`o_proj`	$2048 \to 1024$	2,097,152	49,152
`gate`, `up`	$1024 \to 3072$	$2\times$ 3,145,728	$2\times$ 65,536
`down_proj`	$3072 \to 1024$	3,145,728	65,536
one layer		15,728,640	**360,448**
28 layers		440.4 M	**10.09 M**

Rank $r = 16$. Shapes from the model's configuration: hidden size 1024, 16 query heads and 8 key-value heads of size 128, feed-forward size 3072. Computed for this course.

`q_proj` by hand:
full: $2048 \times 1024 = 2{,}097{,}152$
A: $16 \times 1024 = 16{,}384$
B: $2048 \times 16 = 32{,}768$
LoRA: 49,152, which is 2.3% of the full matrix

- 10.09 M trained parameters out of 596 M: 1.7%. The embeddings and norms stay frozen.
- At 16 bytes each, their training state is 0.16 GB instead of 9.5 GB.

LoRA in Code With PEFT

```
from transformers import AutoModelForCausalLM
from peft import LoraConfig, get_peft_model

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen3-0.6B")
config = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules="all-linear",  # q,k,v,o,gate,
        up,down
    task_type="CAUSAL_LM")
model = get_peft_model(model, config)
model.print_trainable_parameters()
```

```
trainable params: 10,092,544 || all params:
606,142,464 || trainable%: 1.6650
```

- `get_peft_model` wraps each target matrix with its A and B and freezes everything else. `"all-linear"`: every linear layer in the blocks, not the output head.
- The printed count matches the hand count: 10,092,544.
- "all params" now includes the adapters: $596{,}049{,}920 + 10{,}092{,}544$.
- The same three lines work for any model in `transformers`; only the names of the target matrices change between families.

Rank, Alpha and Target Layers: The Three Choices

- **Rank** r: how much the update can express. 8 to 16 is a common start; larger ranks cost more parameters and help only when the task is far from what the model already does.
- **Alpha** α: the update is multiplied by α/r. A common habit is $\alpha = 2r$; if you change r, keep α/r fixed and the learning rate stays valid.
- **Target layers**: the attention matrices only, or every linear layer. Dettmers et al. found LoRA needs all linear layers to match full fine-tuning.

Dettmers et al., NeurIPS 2023, arXiv:2305.14314v1.

LoRA	trained	English	Arabic
$r = 4$, all linear	2.5 M	95%	91%
$r = 16$, all linear	10.1 M	95%	91%
$r = 64$, all linear	40.4 M	**99%**	**94%**
$r = 16$, attention only	4.6 M	97%	92%

Qwen3-0.6B, 20 training requests per intent and language, the same recipe for every row; $\alpha = 2r$. Computed for this course.

- Every setting reached 95 to 99% in English. With 100 test requests a gap of 2 or 3 points can be chance; rank 64 gained 4 points for 4 times the parameters, and attention only did as well as all layers on this narrow task.

Keep the Adapter Apart, or Merge It In

Qwen3-0.6B, LoRA $r = 16$	
adapter file, 32-bit	40 MB
merged model, 16-bit	1.19 GB
time to merge	0.14 s
same answer, merged and unmerged	200 of 200

The adapter trained on 20 requests per intent and language; merged with `merge_and_unload` in 32-bit, then saved in 16-bit. Same answer: greedy decoding on the 200 test requests. Computed for this course.

- **Apart:** one base model and many small adapter files, one per task or customer. A server can switch adapters per request; the adapter can be turned off to get the original model back.
- **Merged:** `model.merge_and_unload()` adds $\frac{\alpha}{r}BA$ into W. One ordinary model, as fast as the original, ready for GGUF and Ollama.
- **QLoRA:** merge into a 16-bit copy of the base, not the 4-bit model, and score again: the adapter was trained on the 4-bit weights. Qwen3-1.7B on a T4: intent 94 to 96% in English, 92 to 89% in Arabic.

QLoRA: A 4-Bit Base With LoRA on Top

- **The problem LoRA leaves:** the frozen weights still sit in memory in 16 bits. Qwen3-8B needs 16.4 GB for them alone: more than a T4.
- **The fix:** store the frozen base in 4-bit NF4 and train 16-bit LoRA adapters on top. Each layer turns its 4-bit weights back into 16 bits for the multiplication, then discards the copy.
- Two more savings from the same paper: **double quantization** (the scales of the groups are quantized too) and **paged optimizers** (optimizer states move to CPU memory when the GPU runs short).
- Reported result: a 65B model fine-tuned on one 48 GB GPU, matching 16-bit fine-tuning.

Dettmers et al., NeurIPS 2023, arXiv:2305.14314v1.

Qwen3-8B on a T4, by hand

	LoRA	QLoRA
frozen base	16.4 GB	6.1 GB
adapters	0.17 GB	0.17 GB
fits in 16 GB?	no	yes

4-bit base: 6.95 B linear weights $\times$ 4.13 bits (NF4 with double-quantized scales), plus the embeddings and output head, which stay 16-bit (2.5 GB). Adapters: 43.6 M parameters at 4 bytes. Activations come on top.

What Trains on a T4, Computed by Hand

	frozen base		LoRA	
model	**16-bit**	**4-bit**	**training**	**fits?**
Qwen3-1.7B	3.4	1.4	0.28	both
Qwen3-4B	8.0	2.7	0.53	both
Qwen3-8B	16.4	6.1	0.70	4-bit only

GB. LoRA training: adapters at rank 16 on all linear layers (17.4 M, 33.0 M and 43.6 M parameters) at 16 bytes each: weights, gradients and AdamW states. 4-bit: NF4 with double-quantized scales; embeddings and output head stay 16-bit. Activations come on top. Computed for this course from the models' configurations.

- The adapters' training state is small for every size: under 1 GB. The frozen base decides what fits.
- On a 16 GB T4: up to 4B with a 16-bit base, if the batch is small; 8B only in 4 bits.
- **Measured on a T4:** Qwen3-1.7B with QLoRA, 8 requests per step: 1.3 GB of 4-bit weights, 5.1 GB at the peak. The rest is activations, which grow with batch size and sequence length; gradient checkpointing stores fewer and recomputes the rest.
- The price of QLoRA is time: every layer turns its weights back into 16 bits at every step.

QLoRA in Code

```
import torch
from transformers import (AutoModelForCausalLM,
                          BitsAndBytesConfig)
from peft import (LoraConfig, get_peft_model,
                  prepare_model_for_kbit_training)

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16)  # T4
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen3-8B", quantization_config=bnb,
    device_map="auto")
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(
    r=16, lora_alpha=32,
    target_modules="all-linear",
    task_type="CAUSAL_LM"))
```

- The only change from LoRA: a `BitsAndBytesConfig` when loading.
- `prepare_model_for_kbit_training` keeps norms in 32 bits and turns on gradient checkpointing (recompute activations instead of storing them).
- A T4 has no bfloat16: compute in float16. On an A100 or newer, use bfloat16.
- `"all-linear"` targets every linear layer except the output head.

Unsloth: The Same Recipe, Less Memory and Time

```
from unsloth import FastLanguageModel

model, tokenizer = (
    FastLanguageModel.from_pretrained(
        "unsloth/Llama-3.2-3B-Instruct",
        max_seq_length=2048, load_in_4bit=True))
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=16, lora_dropout=0,
    target_modules=["q_proj", "k_proj",
        "v_proj", "o_proj", "gate_proj",
        "up_proj", "down_proj"],
    use_gradient_checkpointing="unsloth")
# then TRL's SFTTrainer, as before
```

- **The problem:** QLoRA saves memory but costs time, and a free GPU session is short.
- **Unsloth** rewrites the slow parts of training (attention, LoRA layers, the loss) as faster GPU kernels. Its README: "2× faster with 70% less VRAM".
- The same ideas underneath: a 4-bit base, LoRA adapters, TRL's trainer. Only the loading changes.
- It needs an NVIDIA GPU, "minimum CUDA Capability 7.0": a T4 works; a CPU does not.
- **Measured on a T4:** Llama-3.2-3B in 4 bits, 60 steps of 8 conversations: 9.5 minutes, 3.2 GB of GPU memory at the peak.

Unsloth, github.com/unslothai/unsloth.

SFT Data: One Short Conversation per Example

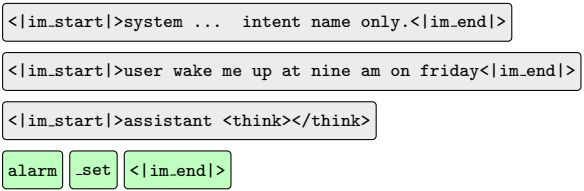
system: Classify the user's request into exactly one of these intents: alarm_set, weather_query, . . . Answer with the intent name only.
user: wake me up at nine am on friday
assistant: alarm_set

system: the same instruction
user: صحيني تسعة الصباح يوم الجمعة
assistant: alarm_set

Two rows of the MASSIVE training set: the same request in English and in Arabic.

- **Supervised fine-tuning (SFT):** train on conversations whose last answer is the one you want.
- Use the same system message, chat template and short prompt that the application will send. The model learns the answer for exactly that input.
- Our set: 10 intents, English and Arabic, 5, 20 or 100 requests per intent and language. The test requests are never in it.
- Labels you already have (tickets, logs, forms) are often enough to build such a set.

Train on the Answer, Not on the Question



grey: label −100, no loss green: the loss is computed here

- **The problem:** with the loss on every token, most of the training signal goes to predicting the instruction and the user's words, which the model will never have to write.
- **The fix:** set the labels of the prompt tokens to −100, the value PyTorch's cross-entropy ignores. Here the loss covers 3 tokens.
- TRL does this by default when the data has a `prompt` and a `completion`. Unsloth calls it `train_on_responses_only`.
- The end-of-turn token is in the loss: the model learns where to stop.

SFT With TRL

```
from datasets import Dataset
from peft import LoraConfig
from trl import SFTConfig, SFTTrainer

rows = [{"prompt": render(system, text),
         "completion": label + "<|im_end|>"}
        for text, label in train_examples]
lora = LoraConfig(r=16, lora_alpha=32,
    target_modules="all-linear",
    task_type="CAUSAL_LM")
args = SFTConfig(output_dir="out",
    num_train_epochs=2, learning_rate=2e-4,
    per_device_train_batch_size=8,
    warmup_steps=0.05, max_length=256)
trainer = SFTTrainer(model=model,
    processing_class=tokenizer, args=args,
    train_dataset=Dataset.from_list(rows),
    peft_config=lora)
trainer.train()
trainer.save_model("intent-adapter")
```

- `render` applies the model's chat template with the generation prompt: the exact text the application will send.
- Passing `peft_config` makes the trainer add the LoRA adapter itself.
- `save_model` writes only the adapter: a few tens of MB.
- This is the code behind every measured result in this section.

The Settings That Matter

setting	ours	usual range
learning rate	2×10^{-4}	10^{-4} to 3×10^{-4} with LoRA; about 10^{-5} for full fine-tuning
epochs	2	1 to 3; more on small sets risks memorizing them
batch size	8	as many as fit; gradient accumulation adds more
maximum length	256 tokens	the longest example, not the model's limit
warm-up	5% of steps	a few percent

- LoRA needs a learning rate about ten times higher than full fine-tuning: the adapters start at zero and have far fewer parameters to move.
- Watch the training loss fall, then score a held-out set. A training loss near zero says the set was memorized, not that the task is solved.
- Change one setting at a time, and keep the test set fixed.

Before and After Fine-Tuning, Measured

Qwen3-0.6B	English	Arabic	prompt tokens
prompting			
intent names only	42%	27%	79
10 most similar examples	93%	92%	270
fine-tuned with LoRA, intent names only			
20 requests per intent	95%	91%	79
100 requests per intent	**99%**	**93%**	79

The same 100 test requests per language for every row; the training requests come from MASSIVE's training split. LoRA $r = 16$ on all linear layers, 2 epochs, 32-bit on 16 CPU cores. Computed for this course.

- With the short 79-token prompt, accuracy went from 42% to 99% in English and from 27% to 93% in Arabic.
- The fine-tuned model beats the best prompt with a prompt 3.4 times shorter, and needs no search for examples.
- The model did not change size: 0.6B parameters plus a 10 M-parameter adapter.

How Many Examples? Measured

per intent	examples	steps	English	Arabic	CPU time
0	0	0	42%	27%	–
5	100	26	91%	81%	2 min
20	400	100	95%	91%	8 min
100	2,000	500	99%	93%	39 min

Examples: per intent $\times$ 10 intents $\times$ 2 languages. 16 CPU cores; a T4 is several times faster. Computed for this course.

- Five examples per intent and language already lift English from 42% to 91% and Arabic from 27% to 81%.
- Each fourfold or fivefold increase in data adds less. Arabic gains most from more data.
- Start small, score the test set, and add data where the errors are: the weakest language, the most confused labels.

What Can Go Wrong After Fine-Tuning

problem	what you see	check or fix
memorizing the set	training loss near zero; the test score stops rising	a held-out set; fewer epochs; more varied examples
forgetting	other requests get worse answers, or the task's format leaks into them	score general requests before and after; keep the adapter separate
a different template at serving time	good scores in training, poor answers in the application	send exactly the trained system message and chat template
test data in the training set	a score that is too good to be true	split before anything else; check for near-duplicates
one language left behind	the average looks fine	report every language separately

Check Everything Else Still Works, Measured

request, no system message	adapter off	adapter on
What is the capital of Japan?	The capital of Japan is **Osaka City**.	Tokyo
How many legs does a spider have?	A spider has **8 legs**.	A spider has 8 legs.
What is 12 times 7?	12 times 7 is 84.	$12 \times 7 = 84$
Suggest a name for a black cat.	A great name for a black cat could be **“Misty”** ...	Black Cat

Qwen3-0.6B with the 100-per-intent adapter, greedy decoding, first 40 tokens. Computed for this course.

- Ten general requests: none was answered with an intent label.
- But the style changed: shorter answers and no markdown, because every training answer was one short word.
- Turning the adapter off gave back the original model exactly: 42% and 27% on the test set again.
- Serve the adapter only on the requests it was trained for.

SFT Copies Answers; It Cannot Say Which Answer Is Better

- **The problem:** for "write a reply to this customer" there are many acceptable answers. SFT needs someone to write the best one, and then teaches the model to copy it word by word.
- People find it much easier to **compare** two answers than to write a perfect one: "A is clearer than B".
- Some qualities are easier to judge than to describe: tone, length, how politely the model says no.
- **The fix:** train on **preferences**: pairs of answers to the same prompt, one marked better.

One preference pair

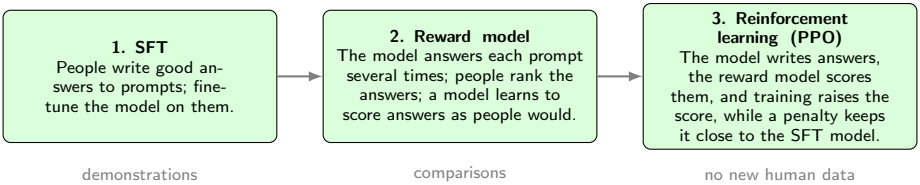
prompt: When was the last time anyone was on the moon?

chosen: The last astronauts left the Moon in December 1972, on Apollo 17.

rejected: As of the latest information available, no human has ever been on the Moon. The Apollo missions, which were the first to land humans on the Moon, ...

The rejected answer is Qwen3-0.6B's own (wrong and long); the chosen one is short and correct.

RLHF in Three Steps



- **RLHF**, reinforcement learning from human feedback, is the recipe behind InstructGPT and the first ChatGPT.
- The result that made it standard: answers of the 1.3B InstructGPT "are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters".

Ouyang et al., NeurIPS 2022, arXiv:2203.02155v1.

Why RLHF Is Hard to Run Yourself

- **Four models in memory** during step 3: the model being trained, a frozen copy for the penalty, the reward model, and a value model that PPO needs to judge partial answers.
- **Generation inside training**: every step samples new answers, which is slow.
- **Many settings**, and runs that can collapse.
- **The reward model is only a stand-in for people.** Optimized too hard, the model finds answers the reward model likes and people do not: "optimizing its value too much can hinder ground truth performance".

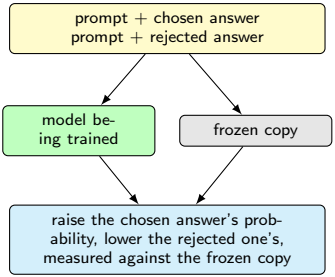
Gao, Schulman and Hilton, ICML 2023, arXiv:2210.10760v1.

	RLHF (PPO)	DPO
models trained	3	1
models in memory	4	2
sampling during training	yes	no
data	pairs + prompts	pairs

Trained: SFT model, reward model, policy (RLHF); policy only (DPO). In memory during the last stage. With LoRA, DPO's frozen copy is the base model with the adapter switched off.

- To tune an open model on your own pairs, DPO is the simpler place to start.

DPO: Learn Directly From the Pairs



- **The fix:** skip the reward model and the reinforcement learning. Train directly on the pairs with a simple classification-style loss.
- The frozen copy keeps the model close to where it started, as the penalty did in RLHF.
- One main setting, β (default 0.1): larger keeps the model closer to the frozen copy.
- The authors: "stable, performant, and computationally lightweight, eliminating the need for sampling from the LM during fine-tuning".

Rafailov et al., NeurIPS 2023, arXiv:2305.18290v3.

Where Preference Pairs Come From

source	how	example
people	show two answers, ask which is better	thumbs up or down in a chat product
a large model as judge	several models answer; a strong model rates each answer	UltraFeedback: 64k prompts, 256k answers, over a million ratings
rules	a program decides: correct beats wrong, valid JSON beats broken, short beats long	our measurement on the next slides
your own model	the model answers twice: with an instruction (chosen) and without (rejected)	teaches the behaviour without the instruction

Cui et al., ICML 2024, arXiv:2310.01377v2.

- Pairs work best when both answers come from the model you train, or one close to it: DPO compares their probabilities under that model.

DPO With TRL

```
from trl import DPOConfig, DPOTrainer

end = "<|im_end|>"
pairs = [{"prompt": render(question),
          "chosen": short + end,
          "rejected": wordy + end}
         for question, short, wordy in data]
args = DPOConfig(output_dir="dpo", beta=0.1,
    learning_rate=5e-5, num_train_epochs=1,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2)
trainer = DPOTrainer(model=model,
    processing_class=tokenizer, args=args,
    train_dataset=Dataset.from_list(pairs),
    peft_config=lora)   # as for SFT
trainer.train()
```

- The data: three text fields per row.
- No reference model is passed: with a LoRA adapter, “the trainer automatically uses the frozen base model as the reference”.
- A lower learning rate than SFT: DPO only nudges a model that already answers well.
- Watch `rewards/accuracies`: how often the chosen answer already scores higher than the rejected one.

Hugging Face TRL documentation, huggingface.co/docs/trl.

DPO, Measured: Short Answers Without Asking

50 questions	words	$\leq$ 15 words	correct
before	61.6	7	7
before, + "Be concise."	11.4	45	8
DPO, lr 10^{-5}	**18.4**	34	**7**
DPO, lr 5×10^{-5}	3.9	50	2
the same, $\beta = 0.5$	6.7	49	3

Qwen3-0.6B with a LoRA adapter, $\beta = 0.1$ unless stated, one epoch on 255 pairs made from 300 Natural Questions (training split); scored on 50 validation questions with no instruction unless stated, greedy decoding. Correct: the answer contains a reference answer, a rough check. Words: mean per answer; lr: learning rate. Computed for this course.

- With the gentle setting, answers fell from 62 to 18 words on average, without any instruction, and kept the same number of correct answers.
- Pushed five times harder, the model learned "shorter wins" too well: four-word answers, and "2010" for the last Moon landing. The pairs taught length only, and length is what it optimized. A larger β held it back only a little.
- Here two words of prompt did as well. Train a behaviour in when the instruction is not under your control, or must hold for every caller; check correctness, not only the trained quality.

Other Preference Methods You Will Meet

method	what it changes	use it when
DPO	pairs; a frozen reference model	the default choice
ORPO	no reference model; SFT and preference in one stage	you start from a base model and want one run
KTO	one answer at a time, marked good or bad; no pairs	your data is thumbs up and thumbs down
SimPO	no reference model; scores by average log-probability	memory is tight

ORPO: Hong et al., EMNLP 2024, arXiv:2403.07691v2; KTO: Ethayarajh et al., ICML 2024, arXiv:2402.01306v5; SimPO: Meng et al., NeurIPS 2024, arXiv:2405.14734v3.

- They differ in what data they need more than in what they can achieve: pick the one that matches the data you have.

When the Answer Can Be Checked, the Reward Is Code

```python
import json, re

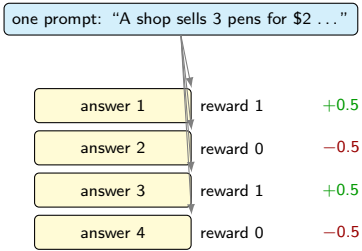
def correct_number(completions, answer, **kw):
    # word problems: compare the last number
    out = []
    for text, gold in zip(completions, answer):
        nums = re.findall(r"-?\d+\.?\d*", text)
        ok = nums and float(nums[-1]) == float(
            gold)
        out.append(1.0 if ok else 0.0)
    return out

def valid_json(completions, **kw):
    out = []
    for text in completions:
        try: json.loads(text); out.append(1.0)
        except ValueError: out.append(0.0)
    return out
```

- **The problem with preferences:** someone, or some model, must judge every pair.
- **When the answer can be checked**, a program judges for free: the right number, code that passes its tests, JSON that parses, a label from the list.
- A reward function takes the model's answers and returns one score per answer. Several functions can be added together.
- DeepSeek-R1-Zero was trained this way, with "accuracy rewards and format rewards" and no reward model.

DeepSeek-AI, arXiv:2501.12948v2.

GRPO: Compare Answers Within a Group



group mean 0.5; each answer’s advantage is its reward minus the mean

- **GRPO**: for each prompt, sample a group of answers (8 by default in TRL), score each, and compare it with the group’s mean.
- Answers above the mean become more likely; answers below become less likely. A penalty keeps the model close to where it started.
- The group mean replaces PPO’s value model: “GRPO foregoes the value model, instead estimating the baseline from group scores”.
- If all answers in a group get the same reward, there is nothing to learn from that prompt: pick prompts the model solves sometimes.

Shao et al., 2024, arXiv:2402.03300v3.

GRPO With TRL

```
from trl import GRPOConfig, GRPOTrainer

# columns "prompt" and "answer"; other columns
# reach the reward functions as keyword arguments
data = load_dataset("openai/gsm8k", "main",
                    split="train").map(to_prompt)
args = GRPOConfig(output_dir="grpo",
    num_generations=8,
    max_completion_length=256,
    learning_rate=1e-6, logging_steps=10)
trainer = GRPOTrainer(model="Qwen/Qwen3-0.6B",
    reward_funcs=[correct_number],
    train_dataset=data, args=args)
trainer.train()
```

- The trainer samples, scores and updates; you write the reward functions and choose the prompts.
- Generation dominates the time: 8 answers per prompt at every step. Libraries such as vLLM and Unsloth make that part faster.
- Log the mean reward and read samples often: a rising reward is not yet a better model.
- **Measured on a T4:** Qwen2.5-1.5B in 4 bits with Unsloth and vLLM, 150 steps of 8 answers, 32 minutes. Answers in the required format: 0 to 99%; correct answers: 60 to 65% of 100 test problems. This run mostly taught the format.

Reward Hacking: The Model Optimizes What You Measure

- **The problem:** the model is rewarded for what the function checks, not for what you meant.
- A robot rewarded for the height of a red block's bottom face "simply flipped over the red block" instead of stacking it.
- Reward models are gamed the same way: past some point, a higher proxy score means lower quality for people.

Krakovna et al., DeepMind blog, 2020, deepmind.google; Gao, Schulman and Hilton, arXiv:2210.10760v1.

A loophole in a reward for our task

Reward: 1 if the correct label appears in the answer.

Hack: answer with all ten labels. Every answer scores 1.

Fix: 1 only if the whole answer is exactly one label, and it is the correct one.

- **Defences:** strict checks; several rewards together (correct and short and well-formed); the penalty for moving far from the start; reading samples during training.

No Labels? Let a Large Model Write Them, Measured

	English	Arabic
teacher, Qwen3-8B, on the test set	96%	94%
teacher's labels for the training set	96%	91%
student, trained on true labels	95%	91%
student, trained on the teacher's labels	**93%**	**86%**

Accuracy. Student: Qwen3-0.6B with LoRA. Teacher prompt: the label definitions. Training set: 20 requests per intent and language (400), labelled once by the teacher; the student recipe is the one used for SFT above. Teacher: 32-bit, thinking off. Computed for this course.

- **The problem:** SFT needs labelled examples, and labelling thousands of requests by hand is slow.
- **The fix:** a large model labels them once; the small model learns from those labels and serves every request after that.
- The teacher's labels were right on 93.5% of the training requests. The student lost 2 points in English and 5 in Arabic against training on true labels: the teacher's mistakes are learned too.
- The student is about 14 times smaller and needs no long prompt. The teacher's cost is paid once; check a sample of its labels by hand, most of all in the weaker language.

Distillation: A Small Model Learns From a Large One

- **The problem:** quantization shrinks how each weight is stored, not how many weights there are. A model with fewer parameters, trained the usual way on text, learns less.
- **The fix: distillation**. Train the small model (the **student**) to match the full output distribution of a large one (the **teacher**), not only the one correct token.
- The teacher's **soft targets** say which wrong answers are nearly right; a one-hot label says nothing about them. Hinton et al. raise the softmax temperature so that the small probabilities carry signal.

Hinton, Vinyals and Dean, "Distilling the Knowledge in a Neural Network," 2015, arXiv:1503.02531v1.

By hand: next word after "The cat sat on the"

	label	teacher	teacher, $T = 2$
mat	1	0.55	0.40
floor	0	0.25	0.27
sofa	0	0.15	0.21
car	0	0.05	0.12

$T = 2$: take $p_i^{1/2}$ and renormalize, the same as dividing the logits by 2. Illustrative probabilities.

The label teaches "mat". The teacher also teaches that "floor" and "sofa" fit and "car" fits poorly.

Distillation in Practice: How Small Models Are Made

- **DistilBERT**: "reduce the size of a BERT model by 40%, while retaining 97% of its language understanding capabilities and being 60% faster".
- **Qwen3**: "For smaller models, we use strong-to-weak distillation, leveraging both off-policy and on-policy knowledge transfer from larger models". The 0.6B to 14B models were taught by the largest ones, with "only 1/10 of the GPU hours" of the full training recipe.
- **Off-policy**: the student learns from text the teacher wrote. **On-policy**: the student writes, and the teacher's probabilities grade every token.

Sanh et al., arXiv:1910.01108v4; Qwen Team, arXiv:2505.09388v1.

For your application, the cheap version:

1. A large model answers a few thousand of your real requests.
2. Keep the answers that pass your checks.
3. Fine-tune a small model on them.
4. Serve the small model, quantized.

- This needs only the teacher's text, so an API model can be the teacher. Check its terms of use first: some forbid training other models on its outputs.

Synthetic Data: Check It Before You Train on It

- **The problem:** a model that writes training data repeats itself and makes mistakes, and the student learns both.
- **Filter:** keep only outputs that pass checks: the label is in the list, the JSON parses, the answer matches a second model or a rule.
- **Deduplicate:** remove near-copies, or one pattern fills the set.
- **Mix:** keep real requests from your users as the inputs; let the large model write only the answers.
- **Keep a human-checked test set** that no model wrote.

Before using a model's outputs as training data

Read the licence of the open model, or the terms of the API. Some forbid using the outputs to train other models; some licences pass conditions on to models trained on them.

- Personal data in the requests stays personal data in the training set: remove it or get consent.

Which Method for Which Goal

goal	method	data you need	measured here
shorter prompts, a narrow task, a fixed format	SFT with LoRA	a few hundred labelled examples	42% → 99%
the same on a bigger model	QLoRA	the same	8B on a T4 in 4 bits
a quality easier to judge than to write	DPO	pairs: better and worse answer	62 → 18 words
answers a program can check	GRPO	prompts and a reward function	–
no labels	a large model labels, a small one learns	unlabelled requests	93% and 86% from 8B labels
knowledge that changes	retrieval, not training	documents	–

- They stack: SFT first, then DPO or GRPO on the SFT model.

Ship It: Merge, Export, Serve

One model, run anywhere

```
model = AutoPeftModelForCausalLM.from_pretrained(
    "intent-adapter", dtype=torch.float16)
model = model.merge_and_unload()  # a new model
model.save_pretrained("intent-merged")
tokenizer.save_pretrained("intent-merged")
```

```
python convert_hf_to_gguf.py intent-merged \
  --outfile intent.gguf --outtype q8_0
# Modelfile: FROM ./intent.gguf
ollama create intent-bot -f Modelfile
```

Many adapters, one base model

```
vllm serve Qwen/Qwen3-0.6B --enable-lora \
  --lora-modules intent=./intent-adapter
```

- **Merged:** an ordinary model folder. Convert it to GGUF and serve it with Ollama, exactly as any downloaded model.
- `merge_and_unload` is "not an in-place operation": keep the model it returns.
- **Apart:** vLLM loads the base once and the adapters beside it. A request picks one "via the model request parameter": `model="intent"`.
- Send the same system message and chat template that the model was trained with.

PEFT documentation, huggingface.co/docs/peft;

vLLM documentation, "LoRA Adapters," docs.vllm.ai.

Before You Ship a Fine-Tuned Model

Checklist

1. The test set was split off first and never trained on.
2. The fine-tuned model beats the best prompt on it, in every language you serve.
3. General requests still get sensible answers, or the adapter is used only for its task.
4. The served prompt and chat template are the trained ones.
5. The merged or quantized model scores the same as the adapter.
6. The data, the base model's licence and the training settings are saved with the adapter.

- Fine-tuning is cheap to repeat. Keep the training script and the data under version control, as you would code.
- When the base model gets a new version, train the adapter again: adapters do not move between models.
- Re-run the test set after every change, including quantization to GGUF.

Summary

- **Decide first:** retrieval for facts that change; a prompt for a quick fix; fine-tuning for a narrow task, a format or a style. Always against a test set and a prompt baseline.
- **LoRA** freezes W and trains BA of rank r: 10.09 M parameters for Qwen3-0.6B at $r = 16$ on all linear layers, 1.7%, counted by hand. Rank 4 to 64, all layers or attention only: 95 to 99% in English on our task.
- **QLoRA** keeps the frozen base in 4-bit NF4 and trains 16-bit adapters. On a 16 GB T4, Qwen3-8B trains only this way: 6.1 GB of frozen weights instead of 16.4 GB.
- **SFT with TRL**, loss on the answer only: with the short 79-token prompt, 42% $\rightarrow$ 99% in English and 27% $\rightarrow$ 93% in Arabic, better than the best 270-token prompt. Five examples per intent already gave 91% and 81%.
- **Check everything else:** no general request was answered with a label, but the style became terse; turning the adapter off restores the original model.
- **RLHF**: SFT, a reward model from human comparisons, then PPO. **DPO** learns from the pairs directly, with one trained model. Trained on 255 pairs, Qwen3-0.6B's answers fell from 62 to 18 words with the same number correct; five times the learning rate gave four-word answers, with 2 correct out of 50 instead of 7.
- **GRPO** rewards checkable answers with Python functions and compares answers within a group. Rewards get gamed: check strictly and read samples.
- **No labels:** Qwen3-8B labelled the training requests (93.5% right); the 0.6B student trained on its labels reached 93% and 86%, against 95% and 91% with true labels.
- **Ship:** keep adapters apart (vLLM serves many on one base) or merge them and export to GGUF for Ollama; re-test after every step.

References

1. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022. arXiv:2106.09685v2
2. Dettmers, Pagnoni, Holtzman and Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," NeurIPS 2023. arXiv:2305.14314v1
3. Gekhman et al., "Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?," EMNLP 2024. arXiv:2405.05904v3
4. Ovadia, Brief, Mishaeli and Elisha, "Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs," 2023. arXiv:2312.05934v3
5. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," NeurIPS 2022. arXiv:2203.02155v1
6. Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," NeurIPS 2023. arXiv:2305.18290v3
7. Gao, Schulman and Hilton, "Scaling Laws for Reward Model Overoptimization," ICML 2023. arXiv:2210.10760v1
8. Cui et al., "UltraFeedback: Boosting Language Models with Scaled AI Feedback," ICML 2024. arXiv:2310.01377v2
9. Hong, Lee and Thorne, "ORPO: Monolithic Preference Optimization without Reference Model," EMNLP 2024. arXiv:2403.07691v2

References (cont.)

10. Ethayarajh et al., "KTO: Model Alignment as Prospect Theoretic Optimization," ICML 2024. arXiv:2402.01306v5
11. Meng, Xia and Chen, "SimPO: Simple Preference Optimization with a Reference-Free Reward," NeurIPS 2024. arXiv:2405.14734v3
12. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," 2024. arXiv:2402.03300v3
13. DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. arXiv:2501.12948v2
14. Krakovna et al., "Specification Gaming: The Flip Side of AI Ingenuity," DeepMind blog, 2020. deepmind.google
15. Hinton, Vinyals and Dean, "Distilling the Knowledge in a Neural Network," 2015. arXiv:1503.02531v1
16. Sanh, Debut, Chaumond and Wolf, "DistilBERT, a Distilled Version of BERT," 2019. arXiv:1910.01108v4
17. Qwen Team, "Qwen3 Technical Report," 2025. arXiv:2505.09388v1
18. FitzGerald et al., "MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages," ACL 2023. arXiv:2204.08582v2

References (cont.)

19. Kwiatkowski et al., “Natural Questions: A Benchmark for Question Answering Research,” TACL 2019; the `nq_open` split, huggingface.co/datasets
20. Hugging Face TRL documentation: SFT, DPO and GRPO trainers, PEFT integration. huggingface.co/docs/trl
21. Hugging Face PEFT documentation: LoRA, merging adapters. huggingface.co/docs/peft
22. vLLM documentation, “LoRA Adapters.” docs.vllm.ai
23. Unsloth. github.com/unslothai/unsloth